

```
# Basic
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Data installation
import os
import requests

# Preprocessing
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import PCA

# Metric
from sklearn.metrics import accuracy_score

# Naive Bayes
from sklearn.naive_bayes import MultinomialNB

# SVM
from sklearn.svm import SVC

# KNN
from sklearn.neighbors import KNeighborsClassifier

# Tree
from sklearn.tree import DecisionTreeClassifier
```

## ✓ Part 1

## ✓ Visualization

```
directory = 'Training'
dirs = [d for d in os.listdir(directory) if os.path.isdir(os.path.join(directory, d))]
```

```
print(dirs)
```

```
['Austen', 'Baum', 'Verne']
```

```
corpus = []
authors = []

for directory in dirs:
    author = os.listdir(f"Training/{directory}")
    for book in author:
        all_words = ''
```

```
# Read files
with open(f"Training/{directory}/{book}", 'r', encoding='utf-8') as f:
    # Reads the entire file as one big string
    all_words = f.read()
    corpus.append(all_words)
    authors.append(directory)

vectorizer = TfidfVectorizer(stop_words='english')
tfidf = vectorizer.fit_transform(corpus)

pca_results = PCA(n_components=2).fit_transform(tfidf.toarray())
```

```
print(authors)
print(pca_results)
```

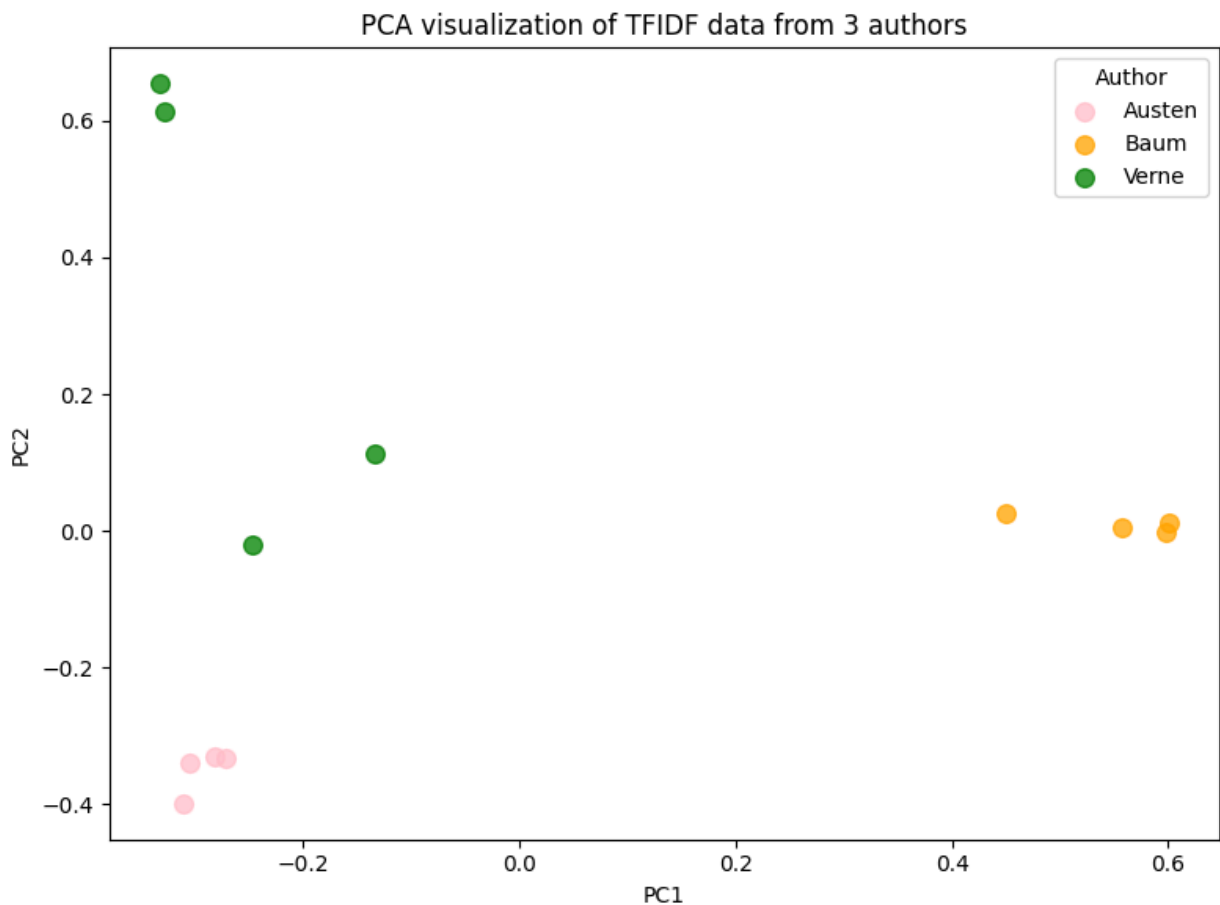
```
['Austen', 'Austen', 'Austen', 'Austen', 'Baum', 'Baum', 'Baum', 'Baum', 'Verne', 'Ve
[[-0.28073346 -0.33018991]
 [-0.27105073 -0.33316677]
 [-0.30439932 -0.33873216]
 [-0.30975137 -0.39868176]
 [ 0.4498929   0.02531968]
 [ 0.60053749  0.01228354]
 [ 0.59846538 -0.00267145]
 [ 0.55761382  0.00430825]
 [-0.3282717   0.61362776]
 [-0.24605514 -0.01924699]
 [-0.33226097  0.65406999]
 [-0.1339869   0.11307983]]
```

```
fig, ax = plt.subplots(figsize=(8, 6))

unique_authors = sorted(set(authors))
colors = {'Austen': 'pink',
          'Baum': 'orange',
          'Verne': 'green'}

for author in unique_authors:
    mask = [l == author for l in authors]
    pts = pca_results[mask]
    ax.scatter(pts[:, 0], pts[:, 1], label=author, color=colors[author], alpha=0.75,

ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.legend(title='Author')
plt.title("PCA visualization of TFIDF data from 3 authors")
plt.tight_layout()
plt.show()
```



## ✓ Prediction

```
# Download the .txt file from the url for each author
# Austen
# url = 'https://www.gutenberg.org/cache/epub/158/pg158.txt'
# response = requests.get(url)

# if response.status_code == 200:
#     with open('Emma.txt', 'wb') as file:
#         file.write(response.content)

# Baum
# url = 'https://www.gutenberg.org/cache/epub/50194/pg50194.txt'
# response = requests.get(url)

# if response.status_code == 200:
#     with open('The_Magic_of_Oz.txt', 'wb') as file:
#         file.write(response.content)

# Verne
# url = 'https://www.gutenberg.org/cache/epub/164/pg164.txt'
# response = requests.get(url)
```

```
# if response.status_code == 200:
#     with open('Twenty_Thousand_Leagues_under_the_Sea.txt', 'wb') as file:
#         file.write(response.content)
```

```
# Building naive bayes model
directory = 'Training'
dirs = [d for d in os.listdir(directory) if os.path.isdir(os.path.join(directory, d))

corpus = []
authors = []

for directory in dirs:
    author = os.listdir(f"Training/{directory}")
    for book in author:
        all_words = ''
        # Read files
        with open(f"Training/{directory}/{book}", 'r', encoding='utf-8') as f:
            # Reads the entire file as one big string
            all_words = f.read()
        corpus.append(all_words)
        authors.append(directory)

vectorizer = TfidfVectorizer(stop_words='english')
tfidf_train = vectorizer.fit_transform(corpus)
```

## ▼ Naive Bayes

```
# Naive bayes
mnb = MultinomialNB().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test') if os.path.isdir(f'Test/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test/{author}'):

        with open(f'Test/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
        tfidf_test = vectorizer.transform([text])
        prediction = mnb.predict(tfidf_test)[0]
        print(f"For {book}, the predicted author is: {prediction}")
        predicted_labels.append(prediction)
        true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")
```

```
For Emma.txt, the predicted author is: Austen
For The_Magic_of_Oz.txt, the predicted author is: Baum
For Twenty_Thousand_Leagues_under_the_Sea.txt, the predicted author is: Verne
Accuracy: 100.0%
```

## ✓ SVM

```
# Support Vector Machine Classifier
svmc = SVC().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test') if os.path.isdir(f'Test/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test/{author}'):

        with open(f'Test/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbc.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")
```

For Emma.txt, the predicted author is: Austen  
For The\_Magic\_of\_Oz.txt, the predicted author is: Baum  
For Twenty\_Thousand\_Leagues\_under\_the\_Sea.txt, the predicted author is: Verne  
Accuracy: 100.0%

## ✓ KNN

```
# KNN Classifier
knnc = KNeighborsClassifier(n_neighbors=3).fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test') if os.path.isdir(f'Test/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test/{author}'):

        with open(f'Test/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbc.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")
```

```
For Emma.txt, the predicted author is: Austen
For The_Magic_of_Oz.txt, the predicted author is: Baum
For Twenty_Thousand_Leagues_under_the_Sea.txt, the predicted author is: Verne
Accuracy: 100.0%
```

## ✓ Decision Tree

```
# Decision tree
tree = DecisionTreeClassifier().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test') if os.path.isdir(f'Test/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test/{author}'):

        with open(f'Test/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbc.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

For Emma.txt, the predicted author is: Austen
For The_Magic_of_Oz.txt, the predicted author is: Baum
For Twenty_Thousand_Leagues_under_the_Sea.txt, the predicted author is: Verne
Accuracy: 100.0%
```

ALL models correctly classified

## ✓ Part2

## ✓ Visualization

## ✓ Download data

```
# Download the .txt file from the url for each author
# url = 'https://www.gutenberg.org/cache/epub/2638/pg2638.txt'
# response = requests.get(url)

# if response.status_code == 200:
#     with open('The_Idiot.txt', 'wb') as file:
#         file.write(response.content)
```

```
directory = 'Training_for_part2'
dirs = [d for d in os.listdir(directory) if os.path.isdir(os.path.join(directory, d))

corpus = []
authors = []

for directory in dirs:
    author = os.listdir(f"Training_for_part2/{directory}")
    for book in author:
        all_words = ''
        # Read files
        with open(f"Training_for_part2/{directory}/{book}", 'r', encoding='utf-8') as f:
            # Reads the entire file as one big string
            all_words = f.read()
        corpus.append(all_words)
        authors.append(directory)

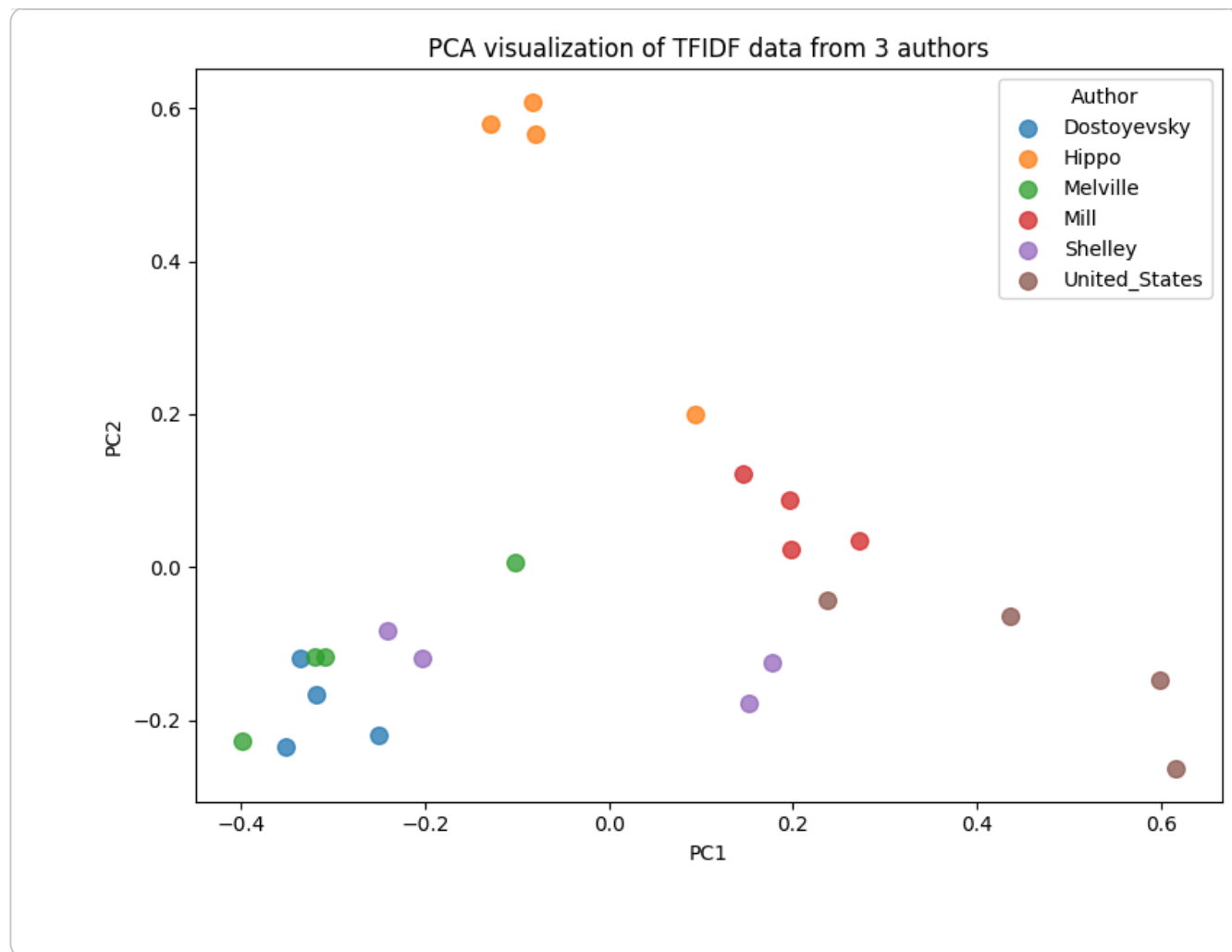
vectorizer = TfidfVectorizer(stop_words='english')
tfidf = vectorizer.fit_transform(corpus)

pca_results = PCA(n_components=2).fit_transform(tfidf.toarray())

# Plot a graph
fig, ax = plt.subplots(figsize=(8, 6))
unique_authors = sorted(set(authors))

for author in unique_authors:
    mask = [l == author for l in authors]
    pts = pca_results[mask]
    ax.scatter(pts[:, 0], pts[:, 1], label=author, alpha=0.75, s=70)

ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.legend(title='Author')
plt.title("PCA visualization of TFIDF data from 3 authors")
plt.tight_layout()
plt.show()
```



- Similar
  - Dostoyevsky
  - Melville
  - Shelley
- Different
  - Hippo
  - United\_States
  - Dostoyevsky

## ✓ Prediction - Similar

```
# Building naive bayes model
directory = 'Training_for_part2'
dirs = ['Dostoyevsky', 'Melville', 'Shelley']

corpus = []
authors = []
```



```

for directory in dirs:
    author = os.listdir(f"Training_for_part2/{directory}")
    for book in author:
        all_words = ''
        # Read files
        with open(f"Training_for_part2/{directory}/{book}", 'r', encoding='utf-8') as f:
            # Reads the entire file as one big string
            all_words = f.read()
        corpus.append(all_words)
        authors.append(directory)

vectorizer = TfidfVectorizer(stop_words='english')
tfidf_train = vectorizer.fit_transform(corpus)

```

```

# Naive bayes
mnb = MultinomialNB().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_part2/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):
        with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnb.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

```

For Crime_and_Punishment.txt, the predicted author is: Dostoyevsky
For The_Confessions_of_St._Augustine.txt, the predicted author is: Shelley
For The_Piazza_Tales.txt, the predicted author is: Melville
For On_Liberty.txt, the predicted author is: Shelley
For Mathilda.txt, the predicted author is: Shelley
For Copyright_Law_of_the_United_States_of_America.txt, the predicted author is: Shelley
Accuracy: 50.0%

```

```

# Support Vector Machine Classifier
svmc = SVC().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_part2/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):

```

```

with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
    text = f.read()
tfidf_test = vectorizer.transform([text])
prediction = mnbc.predict(tfidf_test)[0]
print(f"For {book}, the predicted author is: {prediction}")
predicted_labels.append(prediction)
true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

For Crime\_and\_Punishment.txt, the predicted author is: Dostoyevsky  
 For The\_Confessions\_of\_St.\_Augustine.txt, the predicted author is: Shelley  
 For The\_Piazza\_Tales.txt, the predicted author is: Melville  
 For On\_Liberty.txt, the predicted author is: Shelley  
 For Mathilda.txt, the predicted author is: Shelley  
 For Copyright\_Law\_of\_the\_United\_States\_of\_America.txt, the predicted author is: Shell  
 Accuracy: 50.0%

```

# KNN Classifier
knnc = KNeighborsClassifier(n_neighbors=3).fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_par
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):

        with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbc.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

For Crime\_and\_Punishment.txt, the predicted author is: Dostoyevsky  
 For The\_Confessions\_of\_St.\_Augustine.txt, the predicted author is: Shelley  
 For The\_Piazza\_Tales.txt, the predicted author is: Melville  
 For On\_Liberty.txt, the predicted author is: Shelley  
 For Mathilda.txt, the predicted author is: Shelley  
 For Copyright\_Law\_of\_the\_United\_States\_of\_America.txt, the predicted author is: Shell  
 Accuracy: 50.0%

```

# Desicion tree
tree = DecisionTreeClassifier().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_par
true_labels = []
predicted_labels = []

```

```

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):

        with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbg.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

```

For Crime_and_Punishment.txt, the predicted author is: Dostoyevsky
For The_Confessions_of_St._Augustine.txt, the predicted author is: Shelley
For The_Piazza_Tales.txt, the predicted author is: Melville
For On_Liberty.txt, the predicted author is: Shelley
For Mathilda.txt, the predicted author is: Shelley
For Copyright_Law_of_the_United_States_of_America.txt, the predicted author is: Shell
Accuracy: 50.0%

```

## ✓ Prediction - Different

```

# Building naive bayes model
directory = 'Training_for_part2'
dirs = ['Dostoyevsky', 'Hippo', 'United_states']

corpus = []
authors = []

for directory in dirs:
    author = os.listdir(f"Training_for_part2/{directory}")
    for book in author:
        all_words = ''
        # Read files
        with open(f"Training_for_part2/{directory}/{book}", 'r', encoding='utf-8') as f:
            # Reads the entire file as one big string
            all_words = f.read()
        corpus.append(all_words)
        authors.append(directory)

vectorizer = TfidfVectorizer(stop_words='english')
tfidf_train = vectorizer.fit_transform(corpus)

```

```

# Naive bayes
mnbg = MultinomialNB().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_part2/{d}')]
true_labels = []
predicted_labels = []

for author in test_dirs:

```

```

for book in os.listdir(f'Test_for_part2/{author}'):

    with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
        text = f.read()
    tfidf_test = vectorizer.transform([text])
    prediction = mnbc.predict(tfidf_test)[0]
    print(f"For {book}, the predicted author is: {prediction}")
    predicted_labels.append(prediction)
    true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

```

For Crime_and_Punishment.txt, the predicted author is: Dostoyevsky
For The_Confessions_of_St._Augustine.txt, the predicted author is: Hippo
For The_Piazza_Tales.txt, the predicted author is: Dostoyevsky
For On_Liberty.txt, the predicted author is: Hippo
For Mathilda.txt, the predicted author is: Dostoyevsky
For Copyright_Law_of_the_United_States_of_America.txt, the predicted author is: Unite
Accuracy: 33.33333333333333%

```

```

# Support Vector Machine Classifier
svmc = SVC().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_par
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):

        with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
        tfidf_test = vectorizer.transform([text])
        prediction = mnbc.predict(tfidf_test)[0]
        print(f"For {book}, the predicted author is: {prediction}")
        predicted_labels.append(prediction)
        true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

```

For Crime_and_Punishment.txt, the predicted author is: Dostoyevsky
For The_Confessions_of_St._Augustine.txt, the predicted author is: Hippo
For The_Piazza_Tales.txt, the predicted author is: Dostoyevsky
For On_Liberty.txt, the predicted author is: Hippo
For Mathilda.txt, the predicted author is: Dostoyevsky
For Copyright_Law_of_the_United_States_of_America.txt, the predicted author is: Unite
Accuracy: 33.33333333333333%

```

```

# KNN Classifier
knnc = KNeighborsClassifier(n_neighbors=3).fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_p
true_labels = []

```

```

predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):

        with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbc.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

```

For Crime_and_Punishment.txt, the predicted author is: Dostoyevsky
For The_Confessions_of_St._Augustine.txt, the predicted author is: Hippo
For The_Piazza_Tales.txt, the predicted author is: Dostoyevsky
For On_Liberty.txt, the predicted author is: Hippo
For Mathilda.txt, the predicted author is: Dostoyevsky
For Copyright_Law_of_the_United_States_of_America.txt, the predicted author is: Unite
Accuracy: 33.33333333333333%

```

```

# Desicion tree
tree = DecisionTreeClassifier().fit(tfidf_train, authors)

# Load test file
test_dirs = [d for d in os.listdir('Test_for_part2') if os.path.isdir(f'Test_for_par
true_labels = []
predicted_labels = []

for author in test_dirs:
    for book in os.listdir(f'Test_for_part2/{author}'):

        with open(f'Test_for_part2/{author}/{book}', 'r', encoding='utf-8') as f:
            text = f.read()
            tfidf_test = vectorizer.transform([text])
            prediction = mnbc.predict(tfidf_test)[0]
            print(f"For {book}, the predicted author is: {prediction}")
            predicted_labels.append(prediction)
            true_labels.append(author)
print(f"Accuracy: {accuracy_score(true_labels, predicted_labels) * 100}%")

```

```

For Crime_and_Punishment.txt, the predicted author is: Dostoyevsky
For The_Confessions_of_St._Augustine.txt, the predicted author is: Hippo
For The_Piazza_Tales.txt, the predicted author is: Dostoyevsky
For On_Liberty.txt, the predicted author is: Hippo
For Mathilda.txt, the predicted author is: Dostoyevsky

```